

Fashion-MNIST Classification Under a Parameter Budget: A Compact Convolutional Architecture

Student ID: 10643951

Abstract

The report presents a compact convolutional neural network (CNN) designed for the Fashion-MNIST dataset with a strict parameter limit of 100,000. The final model contains only 83,482 parameters and achieved 92.3% accuracy on the test dataset, meeting the project requirements. The model design strictly adhered to parameter efficiency, architectural simplicity, and robustness to overfitting through batch normalisation, dropout, and a step-based learning rate scheduler. Hyperparameter choices were guided by validation performance on an 80/20 train-validation split. The results demonstrate that a small, carefully tuned CNN can match or outperform much larger architectures while remaining computationally lightweight.

1 Introduction

The goal of this project was to develop an efficient and accurate image classifier for the Fashion-MNIST dataset while ensuring the model remained under 100,000 parameters. Fully connected networks treat images as flat vectors, ignoring spatial relationships that are crucial for texture and shape recognition. Convolutional neural networks (CNNs) offer a practical compromise between representational power and parameter efficiency, making them well-suited to this constraint.

This work details the model architecture, the training and hyperparameter selection process, and the evaluation results obtained on both a held-out validation split and the official marking test set.

2 Model Architecture

2.1 Design Rationale

A CNN architecture was selected for its ability to capture local spatial patterns while sharing weights across the image, resulting in significantly fewer parameters than fully connected networks of comparable accuracy. The architecture was designed to be shallow enough to meet the parameter budget while still expressive enough to exceed the required 88% accuracy threshold.

2.2 Architecture Overview

The final model consists of two convolutional blocks, followed by a dropout layer and a fully connected classifier. A 2x2 max pooling layer reduces the spatial resolution after the second convolution.

- **Conv Block 1:** 32 filters, 3×3 kernel, followed by BatchNorm and ReLU.
- **Conv Block 2:** 48 filters, 3×3 kernel, followed by BatchNorm and ReLU.

- **Max Pooling:** 2×2 downsampling.
- **Dropout:** rate 0.25 applied after flattening.
- **Classifier:** Fully connected layer with 6912 input units and 10 output logits.

After both convolutional layers, the feature map has spatial dimensions of 12×12 and a depth of 48, yielding 6912 features. The fully connected layer contributes the majority of parameters, but careful selection of filter counts and kernel sizes ensures the total remains below 100,000.

Batch normalisation was included to stabilise training and improve convergence, while dropout helps mitigate overfitting in the final representation. ReLU activations were used throughout due to their simplicity and effectiveness.

2.3 Parameter Efficiency

The final parameter count is:

$$\text{Total} = 83,482 < 100,000,$$

Comfortably meeting the constraint. Several wider configurations were tested, but these either exceeded the parameter limit or offered diminishing returns relative to their cost. The chosen architecture, therefore, balances accuracy and efficiency.

3 Training Procedure and Hyperparameter Selection

3.1 Data Processing

The raw Fashion-MNIST images were converted to tensors and normalised to zero mean and unit variance (scaled via $0.5/0.5$). No stochastic data augmentation was used due to the requirement for deterministic transforms during evaluation.

The training dataset was split 80/20 at random, with the validation set used throughout model development.

3.2 Optimisation Strategy

The model was trained using the Adam optimiser with a learning rate of 0.001 and a batch size of 64. Training was conducted for 20 epochs.

A StepLR scheduler was used, reducing the learning rate by a factor of 0.5 every seven epochs. This encourages significant updates early in training and more controlled refinement later on, improving generalisation.

3.3 Hyperparameter Tuning

A manual search was carried out over the following hyperparameters:

- Learning rate: $\{0.01, 0.001, 0.0005\}$
- Batch size: $\{32, 64, 128\}$
- Channel widths: $(16 \rightarrow 32 \rightarrow 64)$, $(32 \rightarrow 48)$, $(32 \rightarrow 64)$
- Dropout rates: $\{0, 0.25, 0.5\}$

- Epoch counts: {10, 15, 20}

The selected configuration (32→48 channels, dropout 0.25, batch size 64, 20 epochs, LR 0.001) consistently provided the best validation accuracy while maintaining fast CPU training times and staying within the parameter limit.

4 Results

4.1 Validation Performance

Training loss decreased smoothly from 0.4368 in epoch 1 to 0.0797 in epoch 20. Validation accuracy improved steadily, reaching a maximum of 92.41%.

This upward trend indicates that the combination of batch normalisation, dropout, and learning rate scheduling successfully prevented overfitting while allowing the model to learn a strong set of features.

4.2 Official Test Performance

Using the marking system’s `model_calls.py` script, the model achieved:

- **Accuracy:** 92.3%
- **Parameter Count:** 83,482
- **Training Check:** Passed

The high test accuracy suggests strong generalisation beyond the validation split and places the model comfortably within the upper tier of submissions.

4.3 Discussion

The main performance bottleneck was achieving high accuracy while staying within the parameter budget. Early prototypes with larger channel counts achieved slightly higher accuracy but came too close to the limit. Meanwhile, smaller models trained quickly but plateaued around 88–90%.

Future improvements could involve experimenting with depthwise separable convolutions, cosine annealing learning rate schedules, or lightweight deterministic data augmentations during evaluation.

5 Conclusion

This project demonstrates that a compact CNN can meet strict parameter constraints while still achieving high accuracy on a widely used benchmark dataset. Through careful feature map sizing, judicious use of batch normalisation and dropout, and targeted hyperparameter tuning, the final model reached 92.3% accuracy with only 83k parameters. The approach balances simplicity, efficiency, and performance, making it suitable for both academic evaluation and deployment in resource-limited environments.

Word Count

Approximately 900 words.